COLLECTION TOTAL — RÉCONCILIATION

L2 — Le SDLC Réconcilié

Partie B : FORGE

La zone où l'intention devient produit éprouvé

PLANIFIER · IMPLÉMENTER

Collection ToTAL — Réconciliation | 21 livres

Images et couverture par Soléane Bélanger

Tous droits réservés — ORIGINALIUM, Disraeli, QC, Canada

Table des matières

L2 — Le SDLC Réconcilié — Partie B : FORGE (PLANIFIER + IMPLÉMENTER)

Rappel de la carte — Situer FORGE dans la trajectoire

La carte à quatre niveaux

Positionnement de FORGE dans le SDLC réconcilié

FORGE comme zone de chevauchement stratégique

Tableau synthèse : SENSE · FORGE · CARE

Chapitre 1 — PLANIFIER : organiser l'apprentissage et la livraison

1.1 La nature de la planification dans ToTAL

1.2 Les objets de planification ToTAL

1.3 Planifier comme stratégie de confiance

1.4 Les dépendances et la capacité

1.5 Scènes DREAM-TEAM / ORIGINALIUM

1.6 BEASTEST : rôle optionnel dans la planification

1.7 Carnets de terrain

1.8 À retenir, Symptômes, Questions, Repères, Signal de vigilance

Chapitre 2 — IMPLÉMENTER : construire avec des preuves progressives

2.1 FORGE : la zone de chevauchement stratégique

2.2 Construire sans perdre le lien avec la valeur

2.3 Preuves progressives : de l'exemple au code

2.4 La dette technique comme risque qualité nommé

2.5 Le décalage à gauche (shift-left) en pratique

2.6 Préparer les signaux pour CARE

2.7 Scènes DREAM-TEAM / ORIGINALIUM

2.8 BEASTEST : rôle optionnel dans l'implémentation

2.9 Carnets de terrain

2.10 À retenir, Symptômes, Questions, Repères, Signal de vigilance

Chapitre 3 — L'ACCEPTATION HOLISTIQUE : au cœur de FORGE

3.1 Qu'est-ce que l'acceptation holistique ?

3.2 Les niveaux d'acceptation dans FORGE

3.3 La définition de « terminé » comme système de confiance

3.4 Scènes DREAM-TEAM

3.5 À retenir, Symptômes, Questions, Repères

Chapitre 4 — FORGE ET LE CENTRE D'EXCELLENCE ToTAL

4.1 Rôle du Centre d'Excellence pendant FORGE

4.2 Gabarits et outillage FORGE

4.3 RACI FORGE

4.4 Métriques de maîtrise pour FORGE

Chapitre 5 — SYNTHÈSE ET CONNEXION AVEC CARE

5.1 Ce que FORGE produit pour CARE

5.2 La transition FORGE → CARE : un passage, pas une rupture

5.3 La grande boucle : CONFIRMER → ÉLICITER (ruban de Möbius)

5.4 Tableau de bord FORGE : les 5 signaux essentiels

5.5 Respiration ToTAL — avant de passer à CARE

Annexes Partie B

Glossaire FORGE

Gabarit Definition of Done (DoD) progressif

Gabarit Plan de Test FORGE

Tableau RACI FORGE

Questions de réflexion pour l'équipe

Quatrième de couverture

Rappel de la carte — Situer FORGE dans la trajectoire

La carte à quatre niveaux

Avant d'entrer dans FORGE, il est utile de rappeler la structure d'ensemble présentée dans la Partie A. Cette carte ne cherche pas à tout expliquer d'un seul regard ; elle cherche à donner à chaque notion sa juste place dans la trajectoire globale.

ToTAL demeure la discipline globale. Elle encadre tout. Sous elle, deux domaines structurent la trajectoire : **SPECIFY** (spécifier) et **DELIVERY** (livrer). Ces deux domaines ne s'opposent pas ; ils se complètent. SPECIFY donne la matière ; DELIVERY donne le mouvement.

À l'intérieur de ces domaines, trois zones de maîtrise organisent le travail en profondeur : **SENSE** (faire sens et ressentir), **FORGE** (forger) et **CARE** (veiller). Chaque zone n'est pas une étape ; c'est un espace de discipline où des compétences spécifiques s'exercent, où des signaux spécifiques circulent, où des décisions spécifiques se prennent.

Enfin, six phases opérationnelles donnent corps à ces zones dans le quotidien des équipes : **ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER**. Ce sont ces phases que les équipes reconnaissent dans leurs outils, leurs rituels, leurs livrables.

> **La carte à quatre niveaux — Rappel synthétique**
>
> **Niveau 1 :** ToTAL — la discipline globale
> **Niveau 2 :** SPECIFY et DELIVERY — les deux domaines
> **Niveau 3 :** SENSE · FORGE · CARE — les zones de maîtrise
> **Niveau 4 :** ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER — les phases opérationnelles

Cette quatrième partie présente une seule carte, à plusieurs niveaux. ToTAL demeure la discipline globale ; SPECIFY et DELIVERY sont deux domaines ; SENSE, FORGE et CARE sont des zones de maîtrise ; ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER et CONFIRMER sont des phases opérationnelles.

Positionnement de FORGE dans le SDLC réconcilié

Le SDLC réconcilié n'est pas un retour au passé : c'est une trajectoire moderne de maîtrise. Il réconcilie ce que les organisations ont souvent séparé : la rigueur de la définition et la souplesse de l'apprentissage. Il ne défend ni le modèle en cascade (waterfall), ni son rejet systématique. Il n'est ni pro-Agile, ni anti-Agile. Il est pro-maîtrise.

Dans cette trajectoire, **FORGE** occupe une position centrale et stratégique. Elle est la zone où les intentions — clarifiées dans SENSE, structurées dans SPECIFY — deviennent réalité construite. C'est ici que le code est écrit, que les composants sont assemblés, que les configurations sont établies, que les systèmes sont éprouvés.

Mais FORGE n'est pas une boîte noire. Elle est traversée par des signaux qui viennent de l'amont (SENSE, SPECIFY) et par des obligations envers l'aval (CARE). Comprendre FORGE, c'est comprendre cette double tension : construire bien, sans perdre le lien avec la valeur qui a justifié la construction.

FORGE comme zone de chevauchement stratégique

Forger (FORGE) est la zone où l'intention devient produit construit, intégré, configuré, monté et éprouvé, sans perdre le lien entre l'amont — besoin, valeur, portefeuille — et l'aval — diffusion, exploitation et signaux opérationnels.

Il faut être précis : FORGE n'est pas « dans l'opérationnel » au sens strict. Elle est principalement ancrée dans le développement, mais elle chevauche le décalage à gauche (shift-left) vers le portefeuille (portfolio) et le décalage à droite (shift-right) vers l'opérationnel. C'est précisément ce chevauchement qui la rend stratégique dans ToTAL.

Ce chevauchement n'est pas un défaut de conception ; c'est une caractéristique délibérée. Les organisations qui isolent complètement leur développement — en amont de toute réalité opérationnelle et en aval de toute contrainte de portefeuille — construisent souvent juste, mais rarement ce qui compte vraiment.

Tableau synthèse : SENSE · FORGE · CARE

Zone	Phases opérationnelles	Question centrale	Ce que la zone produit
SENSE	ÉLICITER ·	Comprenons-nous	Clarté, contexte,

Zone	Phases opérationnelles	Question centrale	Ce que la zone produit
	DÉFINIR	vraiment ce qui compte ?	hypothèses validées, exemples partagés
FORGE	PLANIFIER · IMPLÉMENTER	Construisons-nous ce qui protège la valeur clarifiée ?	Plan organisé, code éprouvé, preuves progressives, signaux de livraison
CARE	DIFFUSER · CONFIRMER	Ce que nous avons construit tient-il dans le réel ?	Signaux opérationnels, apprentissages, décisions d'itération

> **Repère ToTAL**
>
> La rigidité suit les étapes pour se rassurer ; la discipline ToTAL suit une trajectoire pour mieux voir ; la maîtrise ToTAL sait quand avancer, quand revenir et quand apprendre.

CHAPITRE 1

PLANIFIER : organiser l'apprentissage et la livraison

Spécifier rend la livraison planifiable ; planifier rend cette livraison organisable.

Scène d'ouverture — La réunion de planification qui tourne en rond

Lundi matin. La salle de réunion est pleine. Théo a projeté la liste des fonctionnalités du prochain sprint. Rania regarde l'écran en silence. Karim, lui, a déjà son carnet ouvert.

— « On a douze points de capacité, dit Théo. Et j'ai quinze stories. On va devoir décider lesquelles on coupe. »

Rania lève la main doucement. « Avant de décider quoi couper, est-ce qu'on sait laquelle de ces quinze stories va nous apprendre quelque chose si elle échoue ? »

Silence. C'est cette question qui change tout. Planifier, dans ToTAL, commence là : pas dans la liste, mais dans ce qu'on veut apprendre.

1.1 La nature de la planification dans ToTAL

Planifier, dans ToTAL, organise autant l'apprentissage que la livraison. Cette affirmation peut sembler simple ; elle est en réalité profondément différente de ce que la plupart des organisations font quand elles planifient.

Dans la majorité des contextes, planifier signifie : distribuer des tâches dans le temps, allouer des ressources, et s'assurer que les engagements de livraison peuvent être tenus. C'est une planification de la production. Elle est utile, nécessaire même. Mais elle est incomplète si elle n'organise pas aussi les occasions d'apprendre du réel.

Cette phase relie hypothèses, risques, capacité, dépendances et signaux pour préparer la prochaine occasion de sécuriser une valeur ou d'apprendre du réel. Ce qui est planifié n'est donc pas seulement un ensemble de livrables : c'est une stratégie de connaissance.

Ce que planifier n'est PAS dans ToTAL

Planifier organise, il ne remplace pas spécifier (SENSE + SPECIFY) : dans ToTAL, planifier utilise une matière déjà suffisamment clarifiée par faire sens et ressentir (SENSE) et spécifier (SPECIFY). Il ne devrait pas devenir l'endroit où l'on découvre tardivement ce que l'épopée (epic), la livraison (release), la fonctionnalité (feature) ou la capacité voulait réellement dire.

Quand la planification devient le lieu des découvertes tardives, c'est rarement un problème de planification. C'est un signal que SENSE n'a pas suffisamment clarifié, ou que SPECIFY n'a pas suffisamment structuré. Planifier sur une matière floue produit invariablement des plans fragiles, des estimations gonflées et des conversations répétées.

> **Repère ToTAL — La distinction fondamentale**
>
> **Spécifier (SPECIFY)** rend la livraison planifiable. **Planifier** rend cette livraison organisable.
>
> L'une prépare la matière ; l'autre organise le mouvement. Confondre les deux, c'est demander à la planification de faire le travail que la spécification n'a pas terminé.

Planifier autant l'apprentissage que la livraison

Organiser l'apprentissage, c'est décider délibérément : qu'est-ce que cette livraison va nous permettre de savoir que nous ne savions pas avant ? Cette question transforme la planification d'un exercice de remplissage de capacité en une stratégie de construction de confiance.

Une équipe qui planifie l'apprentissage sait qu'une petite livraison bien observée peut apprendre davantage qu'une grande livraison pleine d'hypothèses. Elle sait aussi que certains risques méritent d'être exposés tôt — pas parce qu'on veut échouer, mais

parce qu'un échec rapide et contrôlé coûte infiniment moins qu'un échec découvert en diffusion.

1.2 Les objets de planification ToTAL

Planifier sert surtout à créer, placer ou ajuster les objets de planification et de livraison. Ces objets sont la matière concrète de la planification. Ils ne remplacent pas les spécifications ; ils les *référencent*.

Cette distinction est importante. Un plan de test, par exemple, ne redéfinit pas les comportements attendus ; il organise la manière dont on va vérifier que les comportements définis dans SPECIFY sont bien présents dans le produit construit. Un PI (Program Increment) ne redéfinit pas la vision du portefeuille ; il organise l'effort de plusieurs équipes autour d'objectifs partagés pour une période donnée.

Objet de planification	Rôle dans FORGE	Lien avec les spécifications
Plan de test	Organise la stratégie de vérification et de validation	Référence les exemples et critères définis dans SPECIFY
Plan de validation	Définit les conditions d'acceptation au niveau valeur	Référence les hypothèses de valeur de SENSE
PI (Program Increment)	Cadre l'effort multi-équipes sur une période définie	Ancré dans les objectifs de portefeuille
Release (livraison)	Délimite ce qui sera diffusé ensemble	Référence les fonctionnalités planifiées et leurs critères
Epic (épopée)	Contient une initiative de grande valeur multi-sprints	Décompose en features et stories tracées
Feature (fonctionnalité)	Unité de valeur livrable en un incrément	Reliée aux exemples exécutables de SPECIFY
Story (récit)	Unité de développement au niveau équipe	Critères d'acceptation issus des exemples
Enabler (facilitateur)	Capacité technique qui	Référence les contraintes

Objet de planification	Rôle dans FORGE	Lien avec les spécifications
	soutient la valeur future	architecturales définies
Tâche	Unité de travail individuel à l'intérieur d'une story	Pas directement liée aux spécifications, mais tracée
Billet Jira / outil équivalent	Contenant opérationnel adapté à l'organisation	Dépend du niveau de l'objet qu'il représente

> **Note importante — Adaptabilité des objets**
>
> ToTAL ne prescrit pas un outil de gestion particulier ni une nomenclature rigide. Les organisations utilisent leurs propres termes : épopée, initiative, capacité, carte, incrément. Ce qui compte, c'est que chaque objet soit tracé jusqu'aux spécifications qui lui donnent sens, et jusqu'aux signaux qui confirmeront qu'il a produit sa valeur.

1.3 Planifier comme stratégie de confiance

La planification comme stratégie de confiance : voilà une formulation qui dérange parfois. On associe spontanément la confiance à la qualité du code, à la robustesse de l'architecture, ou à la solidité des tests. Rarement à la planification.

Et pourtant. Une équipe qui livre constamment ce qu'elle a planifié — pas parce qu'elle a gonflé ses estimations, mais parce qu'elle a bien calibré sa capacité, identifié ses dépendances et nommé ses risques — construit une confiance que rien d'autre ne peut remplacer. Cette confiance avec les parties prenantes est le vrai capital de l'équipe sur le long terme.

Livrer plus petit, apprendre plus vite

DREAM-TEAM (équipe) décide parfois de livrer plus petit que prévu, non par manque d'ambition, mais parce qu'une petite livraison bien observée peut apprendre

davantage qu'une grande livraison pleine d'hypothèses. La planification devient alors une stratégie de confiance.

Cette décision — livrer plus petit — est souvent l'une des plus difficiles à prendre dans une organisation sous pression de livraison. Elle exige du courage de la part du PO (Théo), de la clarté de la part du tech lead (Rania), et de la discipline de la part du QA (Karim). Mais elle produit presque toujours de meilleurs résultats : moins de surprises, moins de retravail, plus d'apprentissage par unité de temps investi.

Hypothèses, risques, capacité et dépendances : les quatre piliers du plan

Les hypothèses sont les paris implicites que le plan contient. Un plan sans hypothèses explicitées est un plan sans conscience de ses propres limites. Quand les hypothèses sont nommées, elles peuvent être vérifiées, contestées, et mises à l'épreuve du réel.

Les risques sont les événements qui pourraient compromettre le plan. Un risque nommé peut être géré ; un risque ignoré se transforme en surprise. La planification ToTAL nomme les risques sans catastrophisme et sans déni.

La capacité est ce que l'équipe peut réellement produire dans la période donnée. Elle n'est pas une déclaration d'intention ; c'est une mesure, affinée sprint après sprint, livraison après livraison. Une capacité honnête produit des plans respectés ; une capacité gonflée produit des équipes épuisées et des parties prenantes déçues.

Les dépendances sont les liens entre ce que l'équipe fait et ce que d'autres équipes, systèmes ou parties externes font. Elles sont souvent la source principale des surprises dans un plan. Les identifier tôt, les gérer activement, et ne pas les ignorer : voilà l'une des disciplines les plus importantes de la planification.

1.4 Les dépendances et la capacité

Dépendances visibles et invisibles

Les dépendances visibles sont celles que tout le monde connaît : l'équipe B doit livrer l'API avant que l'équipe A puisse intégrer. Ces dépendances sont gérées — parfois bien, parfois mal, mais elles sont dans les conversations.

Les dépendances invisibles sont bien plus dangereuses. Elles sont invisibles parce que personne ne les a nommées : une hypothèse partagée sur la performance du système, une convention de nommage qui n'a jamais été formalisée, un comportement attendu d'un service externe qui n'a jamais été documenté. Quand ces dépendances se révèlent en cours d'implémentation, elles coûtent très cher.

La planification ToTAL traite les dépendances invisibles avec la même rigueur que les dépendances visibles. Elle pose la question : « Qu'est-ce que nous supposons être vrai ici, et qui nous l'a confirmé ? »

Capacité réelle vs capacité apparente

La capacité apparente est le nombre de personnes multiplié par le nombre d'heures dans le sprint. La capacité réelle tient compte des réunions, des interruptions, des rotations, des apprentissages, des retours à faire sur les sprints précédents, de la dette technique accumulée, et des jours fériés.

Les équipes qui confondent capacité apparente et capacité réelle planifient constamment plus que ce qu'elles peuvent livrer. Elles finissent les sprints avec des stories non terminées, des parties prenantes déçues, et une équipe qui doute d'elle-même.

Éviter le mauvais goulot : ne pas ajouter des ressources au mauvais endroit

L'un des reflexes les plus communs quand une équipe est en retard : ajouter des ressources. Ajouter un développeur, ajouter un testeur, ajouter une réunion. Ce réflexe ignore souvent où se trouve réellement le goulot.

Si le goulot est dans la clarification des exigences, ajouter un développeur n'aidera pas. Si le goulot est dans la revue de code, ajouter un testeur n'aidera pas. Si le goulot est dans les décisions de l'équipe de direction, aucune ressource dans l'équipe n'y changera rien.

La planification ToTAL cherche d'abord à identifier le vrai goulot. Puis elle s'attaque à ce goulot. Cette discipline exige de résister à la pression du « faisons quelque chose » et de s'autoriser le temps de bien voir avant d'agir.

1.5 Scènes DREAM-TEAM / ORIGINALIUM : la planification qui devient stratégie

Scène — La fonctionnalité trop grande

Théo regarde le tableau avec un air préoccupé. La fonctionnalité de gestion des alertes en temps réel — celle que le client attendait depuis deux mois — est estimée à vingt-trois points. La capacité du sprint : douze.

— « On ne peut pas la couper, dit Théo. Le client la veut entière. »

Rania pose son stylo. « Ce que le client veut, c'est savoir si son système réagit bien sous charge. Est-ce qu'on peut lui livrer ça en douze points, sans les vingt-trois ? »

Silence. Karim, qui avait déjà commencé à noter les cas de test, lève les yeux. « Si on livre d'abord le moteur de détection sans l'interface, on peut observer le signal. L'interface, c'est de la valeur d'affichage. Le moteur, c'est de la valeur de décision. »

Théo réfléchit. Puis, lentement : « Vous êtes en train de me dire qu'on peut apprendre l'essentiel en livrant la moitié. »

— « Exactement, dit Rania. Et si le moteur échoue, on aura économisé vingt-trois points d'implémentation d'une interface inutile. »

Ce sprint-là, ils livrent onze points. Et apprennent quelque chose que vingt-trois points ne leur auraient jamais appris : le moteur de détection révèle un problème de latence que personne n'avait anticipé. Le prochain sprint est consacré à résoudre ce problème — avant que l'interface soit visible par qui que ce soit.

Scène — La réunion de planification transformée

Depuis que DREAM-TEAM a adopté les principes ToTAL, les réunions de planification ont changé de texture. Elles ne commencent plus par « combien de points avons-nous ? » mais par « qu'est-ce que nous voulons savoir à la fin de ce sprint que nous ne savons pas encore ? »

Cette question — simple en apparence — restructure tout. Elle force l'équipe à distinguer les stories qui livrent de la valeur mesurable de celles qui livrent de la valeur supposée. Elle force Théo à prioriser en fonction de l'apprentissage, pas seulement de l'urgence perçue. Elle force Karim à définir d'avance comment la preuve de valeur sera observée.

La réunion est toujours aussi longue. Mais à la fin, tout le monde sait non seulement ce qu'ils vont faire, mais pourquoi — et comment ils sauront si ça a fonctionné.

1.6 BEASTEST : peut faciliter la visualisation et l'organisation des objets de planification

Dans certains contextes, un outil comme BEASTEST peut faciliter la visualisation des objets de planification et leur lien avec les spécifications, les exemples et les signaux. Il peut accélérer la traçabilité entre stories, critères d'acceptation et plans de test.

Cependant, BEASTEST demeure toujours optionnel dans ToTAL. Les principes de planification décrits dans ce chapitre s'appliquent indépendamment de l'outil utilisé. Ce qui compte n'est pas l'outil, mais la discipline : organiser autant l'apprentissage que la livraison, tracer les objets jusqu'aux spécifications, et nommer les hypothèses, les risques, la capacité et les dépendances.

> **Note sur BEASTEST**
>
> BEASTEST peut faciliter ou accélérer certaines pratiques de planification. Son utilisation est toujours contextuelle et optionnelle. ToTAL ne le rend jamais obligatoire. Chaque organisation choisit l'outillage qui sert le mieux sa trajectoire de maîtrise.

1.7 Carnets de terrain : planifier sur le terrain

Ce que j'ai observé dans les organisations qui planifient sans apprendre

Dans les organisations où la planification est uniquement une discipline de livraison, un pattern se répète avec une régularité déconcertante. Les plans sont respectés — ou presque. Les livrables arrivent — plus ou moins. Mais six mois après la livraison, personne ne peut dire avec certitude si la fonctionnalité produit réellement la valeur pour laquelle elle a été construite.

Ces organisations ont planifié la construction. Elles ont oublié de planifier l'apprentissage. Résultat : elles construisent vite, mais elles apprennent lentement. Et dans un environnement qui change — ce qui est le cas de la grande majorité des contextes depuis plusieurs décennies — apprendre lentement est une forme de risque stratégique.

L'organisation qui a réconcilié sa planification

Une grande entreprise technologique spécialisée en systèmes embarqués temps réel — avec laquelle j'ai eu l'occasion de travailler — avait un défi particulier : ses sprints étaient planifiés avec précision, mais les fonctionnalités livrées ne produisaient pas les signaux opérationnels attendus. Le code était bon. Les tests passaient. Mais en production, les comportements divergeaient des hypothèses de départ.

Le problème n'était pas dans l'implémentation. Il était dans la planification : les hypothèses de valeur n'avaient jamais été explicitées dans le plan. On planifiait la construction, pas la vérification des hypothèses qui la justifiaient. En nommant les hypothèses comme des objets de planification à part entière — et en planifiant des « fenêtres d'apprentissage » délibérées — l'équipe a progressivement réconcilié sa planification avec son apprentissage.

1.8 À retenir, Symptômes, Questions, Repères, Signal de vigilance

> **À retenir — Planifier dans ToTAL**
>
> - Planifier organise autant l'apprentissage que la livraison.
> - Planifier utilise une matière déjà clarifiée par SENSE et SPECIFY — il ne la produit pas.
> - Spécifier rend la livraison planifiable ; planifier rend cette livraison organisable.
> - Une petite livraison bien observée peut apprendre davantage qu'une grande livraison pleine d'hypothèses.
> - La capacité réelle tient compte de tout ce qui réduit le temps disponible pour la construction.

- Les dépendances invisibles coûtent plus cher que les dépendances visibles.

Symptômes d'une planification déséquilibrée

- Les réunions de planification révèlent régulièrement des ambiguïtés sur ce que la fonctionnalité doit faire.
- Les estimations sont systématiquement sous-évaluées ou gonflées.
- La capacité déclarée ne correspond jamais à la capacité livrée.
- Les dépendances entre équipes sont découvertes en cours de sprint, pas à la planification.
- Personne ne peut dire, à la fin d'un sprint, ce que l'équipe a *appris* (par opposition à ce qu'elle a *livré*).

Questions de réflexion pour l'équipe

- Qu'est-ce que nous voulons apprendre dans ce sprint qui n'était pas clair avant ?
- Quelles hypothèses notre plan contient-il ? Sont-elles nommées ?
- Quels risques pourraient compromettre ce plan ? Sont-ils visibles pour tout le monde ?
- Notre capacité déclarée tient-elle compte de toutes les sources de friction réelle ?
- Nos dépendances sont-elles toutes visibles et confirmées ?

Signal de vigilance — Planifier

Si, à la fin d'une réunion de planification, l'équipe sait ce qu'elle va livrer mais pas ce qu'elle va **apprendre**, la planification n'est qu'une planification de production. Elle organise le mouvement sans orienter l'apprentissage. À terme, elle produit des équipes efficaces mais pas évolutives.

CHAPITRE 2

IMPLÉMENTER : construire avec des preuves progressives

Implémenter ne signifie pas exécuter aveuglément un plan ; cela signifie forger une solution en gardant le lien avec la valeur, les exemples, les risques et les signaux.

Scène d'ouverture — Le code qui passe les tests, mais pas la réalité

Rania regarde le tableau de bord de l'intégration continue. Tout est vert. Tous les tests passent. La couverture est à 87%. Par tous les critères techniques, la fonctionnalité est terminée.

Elle appelle Théo. « Les tests passent. »

Théo : « Parfait. On peut livrer ? »

Rania hésite. « Je ne suis pas sûre. J'ai montré le comportement à Karim ce matin. Il m'a demandé : "si un utilisateur fait ça dans cet ordre, que se passe-t-il ?" Et j'ai réalisé qu'on n'avait pas de test pour cet ordre-là. »

Théo : « Mais c'est un cas limite, non ? »

Rania : « C'est le cas que le client nous avait montré lors de l'atelier SENSE. »

Long silence. La fonctionnalité est techniquement terminée. Mais elle n'est pas encore terminée pour la maîtrise.

2.1 FORGE : la zone de chevauchement stratégique

Forger (FORGE) est la zone où l'intention devient produit construit, intégré, configuré, monté et éprouvé, sans perdre le lien entre l'amont — besoin, valeur, portefeuille — et l'aval — diffusion, exploitation et signaux opérationnels.

FORGE est principalement ancrée dans le développement, mais elle chevauche deux directions simultanément. Vers la gauche (shift-left), elle touche le portefeuille : les décisions d'architecture, les choix de dette technique, les capacités (enablers) qui conditionnent la valeur future. Vers la droite (shift-right), elle touche l'opérationnel : les signaux de livraison qui seront consommés par CARE, les mécanismes d'observabilité, la préparation des environnements.

C'est précisément ce chevauchement qui rend FORGE stratégique. Une zone de développement qui ne regarderait que le code — sans être consciente de l'amont ni de l'aval — construirait juste, mais sans maîtrise.

Shift-left : vers le portefeuille

Le décalage à gauche (shift-left) dans FORGE signifie que les décisions qui affectent la valeur à long terme — architecture, dette technique, capacités transversales — ne peuvent pas être repoussées à la fin du développement. Elles doivent être prises et documentées pendant la construction, en référence aux objets du portefeuille.

Un enabler, par exemple, n'est pas un luxe technique. C'est un investissement dans la capacité future de l'organisation à livrer de la valeur. Quand FORGE gère bien ses

enablers — en les traçant jusqu'aux objectifs du portefeuille — elle construit la fondation de la valeur future, pas seulement la valeur du sprint en cours.

Shift-right : vers l'opérationnel

Le décalage à droite (shift-right) dans FORGE signifie que ce qui sera diffusé doit être préparé pendant la construction. Les mécanismes d'observabilité — logs, métriques, alertes, tableaux de bord — ne s'ajoutent pas après la livraison ; ils se conçoivent et s'implémentent pendant FORGE.

Cette discipline change profondément la manière dont CARE peut faire son travail. Une équipe qui prépare ses signaux de livraison dans FORGE donne à CARE les moyens d'observer ce qui compte. Une équipe qui n'y pense pas laisse CARE dans le noir — contrainte d'interpréter des comportements sans les instruments nécessaires.

2.2 Construire sans perdre le lien avec la valeur

Chez DREAM-TEAM (équipe), une fonctionnalité peut être techniquement terminée mais encore insuffisante pour la maîtrise si le signal produit ne soutient pas la bonne décision. FORGE demande alors : ce que nous avons construit protège-t-il encore la valeur que nous avons clarifiée ?

Cette question n'est pas rhétorique. Elle est opérationnelle. Elle se pose concrètement : les comportements implémentés correspondent-ils aux exemples discutés en SENSE ? Les critères d'acceptation définis en SPECIFY sont-ils tous vérifiés ? Les signaux produits permettront-ils à CARE de mesurer ce qui compte vraiment ?

Le lien amont-développement-aval visible pendant la construction

Dans une organisation sans discipline ToTAL, le développement est souvent une boîte noire. L'amont lui transmet des spécifications ; l'aval en reçoit un livrable. Ce qui se passe à l'intérieur — les décisions, les compromis, les découvertes — reste invisible jusqu'à ce qu'un problème le rende visible.

FORGE rend ce lien visible en permanence. Elle le fait non pas par des rapports d'avancement, mais par des preuves progressives : des tests qui prouvent que le comportement est présent, des exemples exécutables qui montrent que la compréhension est partagée, des signaux de santé qui indiquent que la construction avance dans la bonne direction.

> **Repère ToTAL — Le lien amont-aval**
>
> Le développement ne passe donc pas simplement du code aux tests. Il construit avec des preuves progressives. Plus ces preuves sont reliées aux exemples de SPECIFY et aux signaux de DELIVERY et de CARE, plus FORGE protège la valeur clarifiée sans transformer les tests en étape tardive.

2.3 Preuves progressives : de l'exemple au code

Dans FORGE, la validation ne doit pas attendre la fin du développement. Elle accompagne la construction : exemples exécutables ou discutables, revues, vérifications automatisées, validation de comportement, observation des signaux techniques et préparation des signaux de livraison.

Cette affirmation — la validation accompagne la construction — est au cœur de ce que ToTAL entend par « preuves progressives ». Chaque étape de la construction produit une preuve partielle que la direction est bonne, que la compréhension est partagée, que le comportement attendu commence à émerger.

Exemples exécutables et discutables

Les exemples exécutables sont des scénarios écrits dans un langage compréhensible par tous — développeurs, testeurs, et parties prenantes — et qui peuvent être automatisés. Ils servent à la fois de spécification vivante et de suite de validation. Quand ils passent, ils prouvent que le comportement décrit est présent dans le code.

Les exemples discutables sont ceux qui n'ont pas encore été automatisés, mais qui ont été suffisamment discutés pour que toute l'équipe ait une compréhension partagée du comportement attendu. Ils constituent une forme de connaissance collective qui guide la construction même avant la vérification formelle.

Vérifications automatisées

Les vérifications automatisées — tests unitaires, tests d'intégration, tests de contrat, tests de comportement — sont les instruments de mesure de FORGE. Elles ne remplacent pas le jugement humain ; elles le libèrent de la vérification répétitive pour le concentrer sur l'interprétation et la décision.

Un bon système de vérifications automatisées donne à l'équipe la confiance de modifier le code sans crainte de régressions silencieuses. Cette confiance n'est pas un luxe ; c'est une condition de la vitesse durable.

Validation de comportement

La validation de comportement va au-delà de la vérification technique. Elle pose la question : ce que nous avons construit se comporte-t-il de la manière dont les utilisateurs vont l'utiliser ? Elle implique souvent Théo (PO) dans la revue, et parfois des utilisateurs représentatifs.

Signaux techniques

Les signaux techniques — performance, latence, taux d'erreur, utilisation des ressources — sont des indicateurs de santé du système en construction. Ils ne remplacent pas les signaux de valeur, mais ils alertent quand la construction s'éloigne des contraintes non fonctionnelles qui conditionneront l'expérience en production.

2.4 La dette technique comme risque qualité nommé

La dette technique est inévitable. Ce qui n'est pas inévitable, c'est qu'elle soit invisible. Dans ToTAL, la dette technique est traitée comme un risque qualité nommé — pas comme une honte cachée ni comme une fatalité acceptée en silence.

Rendre visible la dette acceptée

Quand une équipe fait le choix délibéré d'accumuler de la dette technique — pour livrer plus vite, pour explorer une piste, pour répondre à une contrainte externe — ce choix doit être documenté. Qui a décidé ? Pourquoi ? Quand prévoit-on de la rembourser ? Quel est le coût si on ne la rembourse pas ?

Une dette visible peut être gérée. Elle apparaît dans les conversations de planification, dans les estimations futures, dans les décisions de priorisation. Une dette invisible, elle, se manifeste sous forme de « mystérieuses lenteurs », de régressions inexpliquées, et de systèmes qui deviennent de plus en plus coûteux à modifier.

L'effet de la dette technique sur la confiance

La dette technique non gérée érode la confiance à deux niveaux. Au niveau interne, elle érode la confiance de l'équipe en sa propre capacité à maintenir la qualité. Au niveau externe, elle érode la confiance des parties prenantes quand les délais s'allongent sans explication apparente.

En nommant la dette, en la plaçant dans les conversations de planification, et en planifiant délibérément sa gestion, ToTAL réconcilie la nécessité de livrer rapidement avec l'impératif de maintenir une qualité durable.

Quand accepter et quand refuser la dette

ToTAL ne prescrit pas une réponse universelle. Il prescrit une *conversation* explicite. Accepter une dette technique est parfois la bonne décision — quand le coût de la perfection dépasse la valeur de l'apprentissage rapide, quand la piste explorée pourrait être abandonnée. Refuser une dette est parfois la bonne décision — quand elle menace la stabilité d'un système critique, quand elle ralentirait irrémédiablement les prochaines livraisons.

Ce qui n'est jamais la bonne décision : ignorer la question.

2.5 Le décalage à gauche (shift-left) en pratique

TDD / BDD comme discipline de spécification dans le code

Le développement piloté par les tests (TDD) et le développement piloté par le comportement (BDD) sont souvent présentés comme des techniques de test. Dans ToTAL, ils sont d'abord des *disciplines de spécification*. Écrire un test avant le code force à clarifier le comportement attendu avant de chercher à le produire. C'est une forme de shift-left appliquée au niveau du code.

Le BDD va plus loin en ancrant les exemples dans un langage partagé entre développeurs, testeurs et parties prenantes. Quand les scénarios BDD sont écrits en collaboration avec Théo (PO), ils servent à la fois de spécification vivante et de critères d'acceptation vérifiables automatiquement.

Les revues de code comme conversation qualité

Une revue de code réduite à la vérification des normes de style ou de la couverture de tests est une revue de code incomplète. Dans FORGE, la revue de code est aussi une conversation sur la valeur : est-ce que ce code produit le comportement qui était prévu ? Est-ce qu'il le fait de manière à être maintenu durablement ? Est-ce qu'il révèle des décisions qui méritent d'être documentées ?

Karim, en tant que QA de DREAM-TEAM, traite les revues de code comme des conversations qualité — pas comme des audits. Cette posture change la dynamique : au lieu d'une inspection, c'est une exploration partagée.

L'intégration continue comme signal de santé

L'intégration continue (CI) est le pouls de FORGE. Quand elle est verte, elle dit : « ce que nous avons construit jusqu'ici tient ensemble ». Quand elle est rouge, elle dit : « quelque chose s'est brisé ». Plus la fenêtre entre l'introduction du problème et sa détection est courte, moins le coût de la correction est élevé.

DREAM-TEAM traite une CI rouge comme une urgence partagée — pas comme le problème du développeur qui a cassé le build. La responsabilité de la santé de la CI est collective, tout comme la santé du produit qu'elle protège.

2.6 Préparer les signaux pour CARE

Ce que FORGE doit produire pour CARE va au-delà du code fonctionnel. CARE a besoin de signaux pour pouvoir observer, interpréter et décider. Si ces signaux ne sont pas préparés dans FORGE, CARE devra les improviser — avec tous les risques que cela implique.

Les signaux de livraison préparés pendant la réalisation

Un signal de livraison, dans ToTAL, est une indication mesurable qui permet à CARE de savoir si la valeur promise a été produite. Il peut prendre de nombreuses formes : un taux d'adoption, un temps de réponse, un taux d'erreur, un score de satisfaction, une fréquence d'utilisation.

Ce qui est important, c'est que ces signaux soient définis *avant* la livraison — idéalement lors de la planification — et que l'infrastructure qui les capturera soit implémentée dans FORGE. Une fonctionnalité sans signal de livraison défini est une fonctionnalité dont personne ne saura jamais si elle a produit sa valeur.

Ce que FORGE construit	Ce que CARE en fait
Fonctionnalité implémentée et vérifiée	Observe le comportement en production
Signaux techniques (métriques, logs)	Détecte les anomalies et les dégradations
Signaux de livraison définis	Mesure si la valeur promise est produite
Risques nommés et documentés	Surveille les risques prioritaires en opération
Documentation de déploiement	Diffuse en minimisant les incidents
Critères d'acceptation vérifiés	Confirme ou infirme les hypothèses de SENSE

2.7 Scènes DREAM-TEAM / ORIGINALIUM : FORGE en action

Scène — Rania et le signal qui manque

La fonctionnalité de détection d'anomalies est déployée en staging. Tout fonctionne. Les tests passent. L'interface affiche exactement ce qui était spécifié.

Rania convoque une courte réunion. « Avant qu'on marque ça terminé, j'ai une question : comment CARE va savoir si la détection fonctionne en production ? »

Théo : « Les tests ont prouvé que ça marche. »

Rania : « En staging, oui. Mais en production, le comportement du trafic réel est différent. Si le taux de faux positifs monte à 30%, qui va le voir ? Comment ? »

Silence. Personne n'avait défini ce signal.

Karim : « On a trois jours avant le déploiement. Je peux ajouter la métrique de taux de faux positifs et un seuil d'alerte. »

Rania : « Et si on décide que le seuil acceptable est 5%, est-ce que Théo est d'accord avec ça ? »

La conversation dure une heure de plus. Mais au déploiement, CARE dispose du signal exact dont il a besoin pour observer la valeur construite. Ce n'est plus de la supposition — c'est de la maîtrise.

Scène — Karim et la conversation qualité

La revue de code de Karim sur le module d'authentification prend plus

longtemps que prévu. Non pas parce que le code est mauvais — il est impeccable par les standards habituels — mais parce que Karim a une question.

— « Ce comportement ici, dit-il en pointant une ligne, est-ce qu'il correspond à l'exemple qu'on avait discuté avec le client lors de l'atelier ? Celui où un utilisateur revient après une session expirée ? »

Le développeur consulte ses notes. « Je pensais que c'était couvert par le cas de test général de session. »

Karim : « Le cas de test général, oui. Mais pas le comportement spécifique de l'exemple du client. Il voulait que le message soit différent selon le temps d'expiration. »

Deux heures de travail supplémentaires. Une correction mineure. Et une fonctionnalité qui, pour la première fois, respecte exactement l'exemple discuté — pas une approximation technique d'un exemple mal retenu.

Scène — Théo et la pression de livraison

Jeudi soir. La livraison est prévue pour lundi. Le directeur a confirmé la date à trois clients. Rania entre dans le bureau de Théo.

— « On a un problème. La fonctionnalité de rapport passe les tests, mais sous la charge simulée de production, le temps de génération dépasse dix secondes. L'exemple de référence spécifiait moins de trois secondes. »

Théo : « Est-ce que les clients l'utiliseront vraiment sous cette charge ? »

Rania : « C'est l'hypothèse de charge qu'on avait définie ensemble. Je ne peux pas garantir qu'ils ne le feront pas. »

Silence. Théo regarde sa liste d'engagements. Puis : « Qu'est-ce qu'on perd si on livre sans optimiser ? »

Rania : « On livre une fonctionnalité techniquement présente mais

potentiellement inutilisable dans les conditions réelles. Et on crée une dette de performance qu'on devra rembourser plus tard — probablement en urgence, sous pression. »

Théo réfléchit. Il appelle le directeur. La conversation est difficile. Mais la fonctionnalité est reportée de deux semaines. Quand elle est livrée, elle est rapide. Et elle tient la valeur clarifiée.

2.8 BEASTEST : peut faciliter la traçabilité spécification-implémentation-test

Dans certains contextes, BEASTEST peut faciliter la traçabilité entre les exemples définis dans SPECIFY, le code implémenté dans FORGE, et les résultats des vérifications automatisées. Cette traçabilité accélère les revues de qualité et rend visible le lien entre la valeur définie et la valeur construite.

Comme pour toutes les pratiques outillées dans ToTAL, BEASTEST demeure optionnel. La traçabilité peut être maintenue avec des outils plus simples — des commentaires dans le code, des conventions de nommage, des tableaux de bord manuels — tant que la discipline est présente. L'outil facilite ; la discipline protège.

2.9 Carnets de terrain : construire avec maîtrise

La tentation du « terminé » technique sans « confiance » construite

Dans de nombreuses organisations, « terminé » signifie : le code est dans la branche principale, les tests passent, le ticket est fermé. Cette définition de « terminé » est fonctionnelle pour la gestion du flux. Elle est insuffisante pour la maîtrise.

J'ai observé des équipes qui clôturaient des fonctionnalités avec un taux de confiance de 100% selon leurs critères techniques — couverture de tests, builds verts, revues complétées — et qui découvraient, en production, que le comportement réel s'éloignait significativement des hypothèses de départ. Pas parce que le code était mauvais. Parce que le « terminé » technique ne incluait pas la vérification des hypothèses de valeur.

Les organisations qui construisent sans apprentissage

Certaines organisations ont atteint un niveau remarquable d'efficacité dans leur développement : pipelines CI/CD fluides, couverture de tests élevée, déploiements fréquents. Et pourtant, elles n'apprennent pas significativement d'une livraison à l'autre. Elles construisent vite et bien — mais toujours selon les mêmes hypothèses, sans jamais les remettre en question.

Ce paradoxe — l'efficacité sans apprentissage — est l'un des risques les plus subtils du développement moderne. Il est subtil parce que tout semble fonctionner. Les métriques sont bonnes. Les équipes sont productives. Mais la trajectoire de valeur stagne, parce que les hypothèses sous-jacentes ne sont jamais éprouvées.

2.10 À retenir, Symptômes, Questions, Repères, Signal de vigilance

> **À retenir — FORGE et l'implémentation dans ToTAL**
>
> - Implémenter signifie forger une solution en gardant le lien avec la valeur, les exemples, les risques et les signaux.
> - La validation accompagne la construction — elle ne l'attend pas.
> - FORGE chevauche délibérément le portfolio (shift-left) et l'opérationnel (shift-right).
> - La dette technique doit être nommée, documentée et gérée — jamais

ignorée.

- Les signaux de livraison sont préparés dans FORGE pour être consommés par CARE.
- Une fonctionnalité techniquement terminée peut être insuffisante pour la maîtrise.

Symptômes d'une implémentation sans maîtrise

- Les fonctionnalités sont marquées « terminées » mais personne ne sait quel signal elles produiront en production.
- La dette technique s'accumule silencieusement sans jamais apparaître dans les conversations de planification.
- Les revues de code vérifient la forme mais pas le lien avec les exemples de valeur.
- L'intégration continue est ignorée quand elle est rouge — « on la regardera plus tard ».
- Les signaux de livraison pour CARE ne sont définis qu'après le déploiement.

Questions de réflexion pour l'équipe

- Pouvons-nous tracer chaque élément de code jusqu'à l'exemple ou au critère d'acceptation qui le justifie ?
- Notre définition de « terminé » inclut-elle la vérification des hypothèses de valeur ?

- La dette technique que nous avons acceptée ce sprint est-elle visible et documentée ?
- CARE dispose-t-il des signaux nécessaires pour observer ce que nous allons livrer ?
- Nos revues de code sont-elles des conversations qualité ou des audits de conformité ?

Signal de vigilance — FORGE

Si une fonctionnalité est techniquement terminée mais que personne ne sait quelle confiance elle construit, FORGE a perdu une partie du lien avec la maîtrise. La maîtrise n'est pas l'absence de bugs ; c'est la certitude que ce qui a été construit protège encore la valeur qui avait été clarifiée.

CHAPITRE 3

L'ACCEPTATION HOLISTIQUE : au cœur de FORGE

L'acceptation n'est pas une étape finale. C'est une qualité présente tout au long de la construction.

Scène d'ouverture — Quand « accepté » ne suffit pas

La fonctionnalité a passé tous les critères d'acceptation. Théo a signé. Karim a

validé. Les tests d'acceptation automatisés sont au vert.

Pourtant, deux semaines après la livraison, un utilisateur senior soumet un rapport : « Je ne comprends pas comment cette fonctionnalité est censée m'aider. Elle fait ce que vous avez décrit, mais ce n'est pas ce dont j'avais besoin. »

L'équipe se réunit. Ce n'est pas un bug technique. C'est un problème de niveau : les critères d'acceptation avaient été vérifiés au niveau du comportement, mais la validation au niveau de la valeur — est-ce que ça résout réellement le problème de l'utilisateur ? — n'avait jamais été faite.

C'est ce que ToTAL appelle l'acceptation holistique. Et son absence crée exactement ce type de situation.

3.1 Qu'est-ce que l'acceptation holistique ?

L'acceptation holistique est la reconnaissance que « terminé » est un concept multidimensionnel. Il ne suffit pas qu'une fonctionnalité passe les tests techniques pour être acceptée au sens ToTAL du terme. Elle doit être acceptée à plusieurs niveaux, chacun correspondant à une question différente que FORGE se pose pendant la construction.

La différence fondamentale est celle-ci : la **vérification technique** répond à la question « est-ce que ça fonctionne comme spécifié ? » ; la **validation de valeur** répond à la question « est-ce que ça produit la valeur attendue pour l'utilisateur dans son contexte réel ? ». Ces deux questions ne sont pas interchangeables.

L'acceptation holistique n'est pas une étape tardive : c'est une qualité présente tout au long de FORGE. Elle s'exprime dans chaque revue de code, dans chaque discussion d'exemple, dans chaque décision d'architecture. Elle est la conscience permanente que ce qui est construit doit rester aligné avec la valeur qui a justifié sa construction.

3.2 Les niveaux d'acceptation dans FORGE

Niveau	Question centrale	Pratiques associées	Responsable principal
Niveau code	Le code est-il correct et maintenable ?	Tests unitaires, couverture de code, revues de code	Rania (tech lead)
Niveau comportement	Les comportements correspondent-ils aux exemples ?	BDD, exemples exécutables, scénarios discutables	Karim (QA) + Rania
Niveau intégration	Les composants fonctionnent-ils ensemble ?	Tests d'intégration, tests de contrat, intégration continue	Rania + Karim
Niveau système	Le système complet se comporte-t-il correctement ?	Tests de bout en bout, tests de performance, tests de charge	Karim (QA)
Niveau valeur	Ce qui est construit protège-t-il la valeur clarifiée ?	Validation avec Théo (PO), démos, sessions utilisateurs	Théo (PO) + équipe

Niveau code : tests unitaires et couverture

Les tests unitaires vérifient que chaque unité de code — une fonction, une méthode, un module — se comporte correctement en isolation. La couverture de code mesure la proportion du code qui est exercée par ces tests. Ces deux indicateurs sont utiles, mais insuffisants pris isolément.

Un code avec 95% de couverture peut très bien tester les mauvaises choses — les chemins heureux les plus simples — tout en laissant les comportements complexes et les cas limites non vérifiés. ToTAL ne prescrit pas un taux de couverture cible ; il prescrit une réflexion sur ce qui mérite d'être testé et pourquoi.

Niveau comportement : BDD et exemples exécutables

Le BDD (développement piloté par le comportement) traduit les exemples discutés en SENSE en scénarios exécutables qui servent à la fois de spécification et de validation. Quand ces scénarios sont écrits en collaboration avec Théo (PO) et Karim (QA), ils créent une compréhension partagée qui traverse toute la construction.

Niveau intégration : tests d'intégration et contrats

Les tests d'intégration vérifient que les composants fonctionnent correctement ensemble. Les tests de contrat vérifient que les interfaces entre systèmes respectent les engagements attendus. Ces tests sont souvent les plus difficiles à maintenir — et pourtant parmi les plus précieux pour détecter les incompatibilités avant la production.

Niveau système : tests de bout en bout

Les tests de bout en bout vérifient le comportement du système complet tel qu'il sera utilisé en production. Ils sont coûteux à écrire et à maintenir, mais irremplaçables pour s'assurer que tous les niveaux d'acceptation précédents n'ont pas produit un système dont les parties fonctionnent bien individuellement mais mal ensemble.

Niveau valeur : validation avec le PO et les utilisateurs

Le niveau valeur est le plus souvent négligé dans les organisations qui confondent vérification technique et validation de valeur. Il pose la question : est-ce que ce que nous avons construit résout vraiment le problème qui avait été identifié dans SENSE ? Est-ce que l'utilisateur peut l'utiliser de la manière dont il en a besoin ?

Théo (PO) est le gardien de ce niveau dans DREAM-TEAM. Il organise les démos, il sollicite les retours des utilisateurs représentatifs, et il pose inlassablement la question : « Est-ce que ça protège encore la valeur que nous avions clarifiée ensemble ? »

3.3 La définition de « terminé » (done) comme système de confiance

La définition de « terminé » (DoD — Definition of Done) est l'un des outils les plus puissants de FORGE. Quand elle est bien conçue, elle est un système de confiance : elle garantit que tout ce qui quitte FORGE pour aller vers CARE a passé tous les niveaux d'acceptation pertinents.

Terminé pour développer ≠ terminé pour vérifier ≠ terminé pour livrer

Dans ToTAL, il n'y a pas une seule définition de « terminé » ; il y en a plusieurs, correspondant aux différents moments de la trajectoire.

- **Terminé pour développer** : le code est écrit, les tests unitaires passent, la dette technique est documentée.
- **Terminé pour vérifier** : tous les niveaux d'acceptation ont été atteints, les exemples exécutables passent, la validation comportementale est complète.
- **Terminé pour livrer** : les signaux de livraison sont définis, la documentation de déploiement est prête, les risques résiduels sont nommés, CARE a ce qu'il lui faut pour observer.

Cette distinction n'est pas bureaucratique. Elle est opérationnelle. Elle évite les situations où un élément « techniquement terminé » arrive dans CARE sans les instruments nécessaires à son observation.

Comment la DoD évolue avec la maturité ToTAL

Une équipe qui commence sa trajectoire ToTAL peut avoir une DoD simple : tests unitaires, revue de code, déploiement en staging. Au fur et à mesure que sa maturité grandit, la DoD s'enrichit : exemples BDD automatisés, tests de performance, signaux de livraison définis, validation PO complète.

La DoD n'est pas un document figé. C'est un système vivant qui évolue avec la confiance de l'équipe et les apprentissages accumulés dans CARE.

3.4 Scènes DREAM-TEAM : l'acceptation qui change tout

Scène — La démo qui révèle l'écart

Théo anime la démo de fin de sprint. La fonctionnalité fonctionne exactement comme les tests le prévoyaient. Les critères d'acceptation sont tous au vert.

Il invite une utilisatrice senior à essayer. Elle commence par l'étape qu'elle fait en premier dans son travail quotidien — pas l'étape que l'équipe avait imaginée comme point de départ.

En deux clics, elle est bloquée. La fonctionnalité, conçue pour un parcours linéaire, ne gère pas les parcours non linéaires des utilisateurs expérimentés.

Théo ne panique pas. « Est-ce que c'est un comportement que vous attendiez ? » demande-t-il à l'utilisatrice.

— « C'est exactement comme ça que je travaille tous les jours. »

Rania note. Karim note. La fonctionnalité est marquée « validée techniquement, révision de comportement requise ». Elle revient en FORGE le sprint suivant avec un exemple ajouté : le parcours non linéaire de l'utilisatrice expérimentée.

Ce n'est pas un échec. C'est de l'acceptation holistique qui fonctionne comme prévu.

Scène — La DoD qui sauve la livraison

Karim a ajouté une entrée à la DoD de DREAM-TEAM il y a trois sprints : « Les signaux de livraison pour CARE sont définis et testés avant que la fonctionnalité soit marquée terminée pour livrer. »

Au moment du déploiement d'une nouvelle fonctionnalité critique, l'équipe d'opérations réalise que le tableau de bord de surveillance était déjà configuré avec les bonnes métriques — avant même le déploiement. Les seuils d'alerte étaient en place. Les logs étaient structurés selon le format attendu.

Le déploiement se passe sans incident. Deux heures après, une légère dérive de performance est détectée et corrigée en vingt minutes — parce que les signaux étaient là pour la voir.

Un membre de l'équipe d'opérations note : « C'est la première fois en deux ans qu'on reçoit un produit avec tous les instruments déjà en place. »

Karim sourit. Ce n'est pas de la magie. C'est de la discipline.

3.5 À retenir, Symptômes, Questions, Repères

À retenir — L'acceptation holistique dans ToTAL

- L'acceptation n'est pas une étape tardive ; c'est une qualité présente tout au long de FORGE.
- Il y a cinq niveaux d'acceptation : code, comportement, intégration, système et valeur.
- La DoD est un système de confiance — pas une liste de contrôle bureaucratique.
- Terminé pour développer, terminé pour vérifier, et terminé pour livrer sont trois états distincts.
- La validation de valeur — avec le PO et les utilisateurs — est indispensable

à la maîtrise complète.

Symptômes d'une acceptation incomplète

- Les fonctionnalités passent les tests mais échouent dans les démos avec les utilisateurs réels.
- La DoD est identique depuis six mois — elle n'évolue pas avec la maturité de l'équipe.
- Personne ne peut dire à quel niveau d'acceptation une fonctionnalité se trouve actuellement.
- La validation avec le PO est faite après le déploiement, pas avant.

Questions de réflexion pour l'équipe

- Notre DoD couvre-t-elle les cinq niveaux d'acceptation de manière adaptée à notre contexte ?
- Théo est-il impliqué dans la validation de valeur avant que la fonctionnalité soit marquée « terminée pour livrer » ?
- Nos exemples exécutables correspondent-ils aux cas d'usage réels observés chez les utilisateurs ?
- Notre DoD a-t-elle évolué au cours des derniers mois ? Si non, pourquoi ?

CHAPITRE 4

FORGE ET LE CENTRE D'EXCELLENCE ToTAL

Le Centre d'Excellence ne dicte pas ; il guide, outille et révèle ce que la maîtrise rend possible.

4.1 Rôle du Centre d'Excellence pendant FORGE

Le Centre d'Excellence (CdE) ToTAL joue un rôle particulier pendant FORGE. Il n'est pas dans la construction — ce n'est pas son rôle. Il est *autour* de la construction, dans une posture de soutien, de cohérence et de transmission.

Gardien de la cohérence des pratiques de construction

Quand plusieurs équipes travaillent en parallèle dans FORGE, le risque de divergence des pratiques est réel. Une équipe adopte TDD ; une autre le rejette. Une équipe a une DoD rigoureuse ; une autre en a une approximative. Ces divergences ne sont pas nécessairement problématiques à court terme, mais elles créent des frictions à l'intégration et des inégalités de confiance entre les livrables des différentes équipes.

Le CdE maintient une cohérence minimale suffisante : des pratiques de base partagées, des gabarits adaptables, des critères de maturité communs. Il ne standardise pas tout — la diversité contextuelle est une force, pas une faiblesse. Mais il s'assure que les pratiques fondamentales de FORGE sont présentes partout où c'est nécessaire.

Soutien à l'adoption des preuves progressives

Les preuves progressives — TDD, BDD, intégration continue, revues qualité — sont des disciplines qui s'adoptent progressivement. Elles demandent du temps, de la pratique, et souvent du coaching. Le CdE offre ce coaching : il identifie les équipes qui

en ont besoin, il envoie des coachs techniques, il organise des sessions de pratique partagée.

Formation et coaching des équipes

Le CdE organise la formation continue des équipes dans FORGE. Non pas des formations ponctuelles déconnectées du contexte réel, mais des formations ancrées dans les problèmes actuels des équipes. Un atelier BDD organisé au moment où une équipe commence une nouvelle fonctionnalité complexe a infiniment plus d'impact qu'un atelier générique planifié six mois à l'avance.

4.2 Gabarits et outillage FORGE

Gabarit de Definition of Done (DoD) — Structure de base

Niveau	Critères de base	Critères avancés (maturité élevée)
Code	Tests unitaires écrits et passants ; revue de code complétée ; dette technique documentée	Couverture de code > seuil défini par l'équipe ; analyse statique sans avertissements critiques
Comportement	Critères d'acceptation vérifiés manuellement ; démo faite avec le PO	Exemples BDD automatisés et passants ; cas limites documentés
Intégration	Tests d'intégration passants en environnement de staging	Tests de contrat API passants ; pipeline CI/CD vert
Système	Tests de bout en bout couvrant les scénarios principaux	Tests de performance dans les seuils définis ; tests de charge validés
Valeur	Validation PO complétée avant livraison	Signaux de livraison définis et instrumentés ; documentation de déploiement prête

Gabarit de Plan de Test FORGE — Éléments essentiels

Élément	Contenu attendu
Objectif du test	Quelles hypothèses ce plan cherche-t-il à vérifier ou à infirmer ?
Scope	Quelles fonctionnalités, comportements et niveaux sont couverts ?
Approche	TDD, BDD, tests manuels, tests automatisés — quelle combinaison et pourquoi ?
Exemples de référence	Lien vers les exemples définis dans SPECIFY qui guident la construction
Critères de réussite	Comment sait-on que le test a réussi ? Quels seuils, quels comportements ?
Risques identifiés	Quels comportements ou cas limites présentent un risque particulier ?
Signaux pour CARE	Quels signaux ce plan de test prépare-t-il pour l'observation en production ?
Responsables	Qui est responsable de chaque niveau de test ?

Gabarit de Revue de Code orientée maîtrise

Une revue de code orientée maîtrise va au-delà de la vérification syntaxique. Elle pose cinq questions fondamentales :

1. Ce code correspond-il à un exemple ou à un critère d'acceptation défini ? Lequel ?
2. La dette technique introduite (si applicable) est-elle documentée et justifiée ?
3. Les comportements testés couvrent-ils les cas limites discutés en SENSE ?
4. Ce code est-il maintenable par quelqu'un qui n'a pas participé à sa création ?
5. Prépare-t-il correctement les signaux dont CARE aura besoin ?

4.3 RACI FORGE

Activité FORGE	Rania (Tech Lead)	Théo (PO)	Karim (QA)	Organisation / CdE
Planification du sprint	Responsable	Responsable	Consulté	Informé
Gestion de la capacité	Responsable	Informé	Consulté	Informé
Identification des dépendances	Responsable	Consulté	Consulté	Consulté
Implémentation du code	Responsable	Informé	Consulté	Informé
Revues de code	Responsable	Informé	Responsable	Informé
Tests et vérifications	Consulté	Informé	Responsable	Informé
Validation de valeur (PO)	Consulté	Responsable	Consulté	Informé
Gestion de la dette technique	Responsable	Consulté	Consulté	Informé
Préparation des signaux CARE	Responsable	Consulté	Responsable	Informé
Mise à jour de la DoD	Consulté	Consulté	Responsable	Consulté
Cohérence des pratiques CdE	Consulté	Informé	Consulté	Responsable

R = Responsable de l'exécution · C = Consulté · I = Informé. Un seul responsable par activité pour éviter l'ambiguïté.

4.4 Métriques de maîtrise pour FORGE

Couverture de tests — mais pas comme objectif aveugle

La couverture de tests est une métrique utile quand elle est interprétée correctement. Elle mesure la proportion du code exercée par les tests — pas la qualité de ces tests. Une couverture de 80% avec des tests pertinents est plus précieuse qu'une couverture de 100% avec des tests qui ne vérifient que les chemins heureux.

ToTAL recommande de suivre la couverture comme indicateur de tendance, non comme objectif absolu. Une couverture qui diminue d'un sprint à l'autre sans justification est un signal d'alerte. Une couverture stable associée à des tests bien conçus est une indication de discipline.

Taux de défauts trouvés dans FORGE vs dans CARE

Cette métrique est l'une des plus révélatrices de la maturité de FORGE. Elle mesure la proportion des défauts détectés *avant* la diffusion (dans FORGE) par rapport à ceux détectés *après* la diffusion (dans CARE). Plus cette proportion penche vers FORGE, plus l'équipe construit avec des preuves progressives efficaces.

Une équipe qui trouve 80% de ses défauts dans FORGE protège ses utilisateurs. Une équipe qui trouve 80% de ses défauts dans CARE fait porter le coût de la découverte à ses utilisateurs — et à sa réputation.

Temps de cycle (cycle time) de FORGE

Le temps de cycle mesure le temps moyen entre le moment où une story entre dans le développement et le moment où elle est marquée « terminée pour livrer ». Un temps de cycle stable et court indique une équipe qui maîtrise sa construction. Un temps de cycle qui s'allonge progressivement est souvent le signe d'une accumulation de dette technique non gérée ou d'une augmentation de la complexité des dépendances.

Dette technique visible

La métrique la plus simple et souvent la plus ignorée : combien d'éléments de dette technique sont actuellement visibles, documentés, et présents dans les conversations de planification ? Une organisation qui ne peut pas répondre à cette question ne gère pas sa dette — elle la subit.

Métrique	Ce qu'elle mesure	Signal d'alerte
Couverture de tests	Proportion du code exercée par les tests	Baisse progressive sans justification
Taux défauts FORGE vs CARE	Où les problèmes sont-ils détectés ?	> 40% détectés dans CARE
Temps de cycle FORGE	Vitesse de construction avec maîtrise	Allongement progressif sur 3+ sprints
Dette technique visible	Capacité à nommer et gérer la dette	Dette non documentée, non planifiée
Taux de CI rouge	Santé de l'intégration continue	CI rouge > 10% du temps sur une période

---

CHAPITRE 5

SYNTHÈSE ET CONNEXION AVEC CARE

Ce que FORGE construit n'est pas seulement un produit. C'est une promesse préparée pour le réel.

Scène d'ouverture — Le passage

La démonstration finale est terminée. Théo a validé. Karim a confirmé que tous

les niveaux de la DoD sont atteints. Rania a vérifié que les signaux sont en place.

C'est le moment du passage — de FORGE vers CARE. Mais ce n'est pas un passage abrupt. C'est une transition préparée depuis le premier jour du sprint. CARE reçoit non pas une boîte noire, mais un produit éprouvé, instrumenté, documenté, avec ses risques nommés et ses signaux activés.

Rania se tourne vers l'équipe : « Ce qu'on a construit est solide. Maintenant, c'est le réel qui va nous dire si notre compréhension était juste. »

C'est exactement ça, la maîtrise ToTAL.

5.1 Ce que FORGE produit pour CARE

FORGE ne se termine pas par la fermeture d'un ticket ou la fusion d'une branche. Elle se termine quand tout ce dont CARE a besoin pour faire son travail est en place. Cette définition de la fin de FORGE est intentionnellement orientée vers l'aval.

Les signaux que CARE va consommer

CARE a besoin de voir pour décider. Pour voir, il a besoin de signaux. Ces signaux — métriques de performance, logs structurés, alertes configurées, tableaux de bord préparés — sont le vrai « dernier kilomètre » de FORGE. Sans eux, CARE navigue à vue.

La promesse de diffusion préparée dans FORGE

La diffusion (DIFFUSER) est la première phase de CARE. Son succès dépend en grande partie de ce que FORGE lui a préparé : procédures de déploiement documentées, environnements validés, scripts de mise en place testés, plans de retour arrière (rollback) prêts. Une diffusion bien préparée dans FORGE est une diffusion qui se passe sans surprise.

Les risques nommés et documentés

Tout ce que FORGE n'a pas réussi à résoudre complètement — les risques résiduels, les hypothèses non encore vérifiées, les comportements dont on n'est pas sûrs sous certaines conditions — doit être nommé et transmis à CARE. Non pas comme un aveu d'échec, mais comme un acte de responsabilité. CARE ne peut pas surveiller ce qu'il ne sait pas chercher.

5.2 La transition FORGE → CARE : un passage, pas une rupture

Dans un modèle en cascade (waterfall), la transition entre les phases est une rupture : une équipe termine son travail, le passe à la suivante, et passe à autre chose. Dans ToTAL, la transition de FORGE à CARE est un passage continu — une transmission consciente, préparée, et accompagnée.

Les équipes matures organisent cette transition avec des rituels dédiés : une session de « briefing CARE » où l'équipe de développement présente à l'équipe opérationnelle ce qu'elle a construit, comment ça fonctionne, quels sont les risques connus, et où regarder en premier si quelque chose ne va pas.

Cette session n'est pas une cérémonie bureaucratique. C'est la transmission du savoir de construction vers le savoir d'exploitation — l'un des passages les plus importants dans la chaîne de maîtrise ToTAL.

> **Chaîne de maîtrise ToTAL**
>
> **Clarté → Qualité → Confiance → Excellence durable**
>
> SENSE crée la clarté. FORGE construit la qualité. CARE confirme la confiance. La trajectoire complète produit l'excellence durable.

5.3 La grande boucle : CONFIRMER → ÉLICITER (ruban de Möbius)

ToTAL n'est pas une trajectoire linéaire qui se termine avec CONFIRMER. CONFIRMER est le point où l'apprentissage du réel revient enrichir ÉLICITER — la première phase de la prochaine itération. C'est la grande boucle, représentée dans ToTAL par l'image du ruban de Möbius : toujours en avant, jamais figé, sans retour en arrière au sens strict.

Ce que CARE apprend dans la diffusion et la confirmation — les comportements réels des utilisateurs, les signaux opérationnels, les hypothèses infirmées ou confirmées — devient la matière première d'ÉLICITER dans le prochain cycle. L'organisation qui maîtrise cette grande boucle ne répète pas ses erreurs ; elle les transforme en apprentissages qui guident les prochaines décisions.

L'apprentissage de CARE qui revient enrichir FORGE dans la prochaine itération est l'une des formes les plus précieuses de l'amélioration continue. Il ne s'agit pas d'améliorer les processus en abstrait ; il s'agit d'améliorer la compréhension concrète de ce qui compte, de ce qui fonctionne, et de ce qui reste à résoudre.

5.4 Tableau de bord FORGE : les 5 signaux essentiels

#	Signal	Ce qu'il révèle	Quand il est préoccupant
1	Taux de fonctionnalités terminées pour livrer / planifiées	La fidélité du plan à la réalité de la construction	< 70% de façon répétée
2	Taux de défauts détectés dans FORGE vs CARE	L'efficacité des preuves progressives	> 40% détectés en CARE
3	Temps de cycle moyen (code →	La fluidité et la maîtrise de la	Allongement progressif sur 3+

#	Signal	Ce qu'il révèle	Quand il est préoccupant
	terminé pour livrer)	construction	sprints
4	Éléments de dette technique nommés et planifiés	La visibilité et la gestion de la dette	Nombre stable depuis 2+ mois sans réduction
5	Signaux de livraison définis avant déploiement	La préparation de CARE dans FORGE	< 80% des fonctionnalités avec signaux définis

5.5 Respiration ToTAL — avant de passer à CARE

Avant de refermer cette Partie B et d'ouvrir la Partie C (CARE), une pause s'impose.

FORGE est exigeante. Elle demande une discipline de construction qui maintient simultanément la rigueur technique et le lien avec la valeur. Elle demande à l'équipe de voir loin — vers l'aval — tout en gardant les pieds dans le présent de la construction. Elle demande du courage : le courage de nommer la dette, de reporter une livraison insuffisante, de préparer les signaux même quand le temps manque.

Cette exigence n'est pas une punition. C'est la définition de la maîtrise. Et la maîtrise, dans ToTAL, n'est pas réservée aux équipes d'élite ou aux organisations avec des budgets illimités. Elle est accessible à toute équipe qui décide de regarder honnêtement ce qu'elle construit, de nommer ce qu'elle ne sait pas, et de travailler délibérément à réduire l'écart entre l'intention et la réalité.

> **Ce que FORGE enseigne, au fond**
>
> La rigidité suit les étapes pour se rassurer. La discipline ToTAL suit une trajectoire pour mieux voir. La maîtrise ToTAL sait quand avancer, quand revenir et quand apprendre. FORGE est la zone où la discipline de la construction

> rencontre la discipline de la valeur. Ce n'est pas une étape à traverser — c'est une zone à habiter avec conscience.

FORGE se ferme. CARE s'ouvre.

Ce que vous avez construit va maintenant rencontrer le réel.

Et le réel va vous apprendre ce que la construction ne pouvait pas encore vous dire.

Annexes — Partie B : FORGE

Annexe A — Glossaire FORGE (termes clés avec définitions opérationnelles)

Terme	Définition opérationnelle dans le contexte ToTAL / FORGE
Acceptation holistique	Processus continu de vérification et de validation à plusieurs niveaux (code, comportement, intégration, système, valeur) présent tout au long de FORGE, pas seulement à la fin.
BDD (Behaviour-Driven Development)	Pratique de développement dans laquelle les comportements attendus sont définis en langage partagé (Given/When/Then) avant d'être implémentés, servant à la fois de spécification et de validation automatisée.
Capacité réelle	Ce qu'une équipe peut réellement produire dans une période donnée, en tenant compte de toutes les sources de friction (réunions, interruptions, dette technique, rotations, etc.).

Terme	Définition opérationnelle dans le contexte ToTAL / FORGE
CI/CD (Intégration et livraison continues)	Pipeline automatisé qui détecte rapidement les problèmes d'intégration et prépare les artefacts de déploiement. Signal de santé permanent de FORGE.
Décalage à droite (shift-right)	Extension des préoccupations de FORGE vers l'exploitation opérationnelle : préparation des signaux, observabilité, documentation de déploiement.
Décalage à gauche (shift-left)	Extension des préoccupations de FORGE vers le portefeuille : décisions d'architecture, gestion des enablers, qualification des dépendances.
Definition of Done (DoD)	Ensemble des critères qu'une fonctionnalité doit satisfaire pour être considérée terminée à chaque niveau (développement, vérification, livraison). Système vivant qui évolue avec la maturité de l'équipe.
Dette technique	Ensemble des compromis techniques acceptés délibérément ou par accumulation, qui augmentent le coût futur du changement. Dans ToTAL, elle doit être visible, documentée et gérée activement.
Enabler (facilitateur)	Travail technique qui ne livre pas de valeur directe à l'utilisateur mais qui crée la capacité de livrer de la valeur future (architecture, infrastructure, exploration technique).
Epic (épopée)	Conteneur de planification qui regroupe un ensemble de fonctionnalités et stories partageant un objectif de valeur commun, généralement sur plusieurs sprints ou releases.
Exemples exécutables	Scénarios écrits en langage partagé, ancrés dans les comportements discutés en SENSE, automatisables et servant à la fois de spécification vivante et de validation.

Terme	Définition opérationnelle dans le contexte ToTAL / FORGE
FORGE	Zone de maîtrise ToTAL correspondant aux phases PLANIFIER et IMPLÉMENTER. Zone principale de développement qui chevauche délibérément le portefeuille (shift-left) et l'opérationnel (shift-right).
PI (Program Increment)	Cadre de planification multi-équipes sur une période définie (généralement 8 à 12 semaines), ancré dans les objectifs de portefeuille et décomposé en sprints d'équipe.
Preuve progressive	Ensemble des vérifications et validations produites pendant la construction, qui montrent que la direction est bonne sans attendre la fin du développement.
Signal de livraison	Indicateur mesurable défini avant la diffusion qui permettra à CARE d'observer si la valeur promise est produite en conditions réelles.
TDD (Test-Driven Development)	Pratique de développement dans laquelle le test est écrit avant le code, forçant la clarification du comportement attendu avant l'implémentation.
Temps de cycle (cycle time)	Durée moyenne entre l'entrée d'une story dans le développement actif et son marquage « terminé pour livrer ». Indicateur de fluidité et de maîtrise de la construction.

Annexe B — Gabarit Definition of Done (DoD) progressif

Ce gabarit est adapté pour les organisations en début de trajectoire ToTAL. Il définit trois niveaux de maturité. Chaque équipe commence au niveau 1 et progresse selon son rythme d'adoption.

Critère	Niveau 1 (Base)	Niveau 2 (Consolidé)	Niveau 3 (Maîtrise)
Tests unitaires	Écrits et passants	Couvrent les cas nominaux et les cas d'erreur principaux	Couvrent les cas limites discutés en SENSE
Revue de code	Au moins un relecteur	Conversation qualité documentée	Lien avec les exemples de SPECIFY vérifié
Exemples	Critères d'acceptation vérifiés manuellement	Scénarios BDD écrits et discutés	Scénarios BDD automatisés et passants
Intégration	Build CI vert	Tests d'intégration passants en staging	Tests de contrat API passants
Validation PO	Démo faite avec le PO	PO a validé les critères d'acceptation	Validation avec utilisateur représentatif incluse
Dette technique	Non applicable en niveau 1	Dette introduite documentée	Dette planifiée pour remboursement
Signaux CARE	Non applicable en niveau 1	Signaux de livraison identifiés	Signaux de livraison implémentés et testés
Documentation	Code commenté aux endroits complexes	Décisions architecturales notées	Runbook de déploiement prêt

Annexe C — Gabarit Plan de Test FORGE

Section	Contenu à compléter
Fonctionnalité / Story	[Identifiant et titre de la fonctionnalité ou de la story]
Objectif du test	[Quelles hypothèses ou

Section	Contenu à compléter
	comportements ce plan cherche-t-il à vérifier ?]
Référence aux exemples SPECIFY	[Lien ou référence vers les exemples définis dans la phase SPECIFY]
Scope	[Ce qui est inclus / Ce qui est exclu de ce plan de test]
Approche de test	[TDD / BDD / Tests manuels / Tests automatisés — justification du choix]
Niveaux couverts	[Code ☐ Comportement ☐ Intégration ☐ Système ☐ Valeur ☐]
Cas nominaux	[Liste des scénarios de fonctionnement normal à tester]
Cas limites et d'erreur	[Liste des cas limites et comportements d'erreur à tester]
Risques identifiés	[Comportements ou conditions présentant un risque particulier]
Critères de réussite	[Conditions précises qui indiquent que le test a réussi]
Signaux pour CARE	[Signaux de livraison que ce plan prépare pour l'observation en production]
Responsables	[Qui est responsable de chaque niveau de test]
Date de complétion prévue	[Date cible pour la complétion du plan de test]

Annexe D — Tableau RACI FORGE (version complète)

Activité	Tech Lead	PO	QA	Développeur	CdE	Direction
Planification du sprint	R	R	C	C	I	I

Activité	Tech Lead	PO	QA	Développeur	CdE	Direction
Identification dépendances	R	C	C	C	C	I
Gestion capacité	R	I	C	C	I	I
Implémentation code	R	I	C	R	I	—
TDD / BDD	R	C	R	R	C	—
Revue de code	R	I	R	C	I	—
Intégration continue	R	I	C	R	C	—
Validation de valeur	C	R	C	I	I	I
Gestion dette technique	R	C	C	C	C	I
Préparation signaux CARE	R	C	R	C	I	I
Mise à jour DoD	C	C	R	C	C	I
Cohérence pratiques	C	I	C	I	R	I

R = Responsable de l'exécution · C = Consulté avant la décision · I = Informé après la décision · — = Non impliqué

Annexe E — Questions de réflexion pour l'équipe

Ces questions sont conçues pour alimenter les rétrospectives, les sessions de coaching et les conversations d'équipe pendant FORGE. Elles ne sont pas des audits ; elles sont des invitations à regarder honnêtement la trajectoire.

Sur la planification

6. Notre dernier sprint a-t-il organisé l'apprentissage autant que la livraison ? Comment le savons-nous ?
7. Quelles hypothèses notre plan de sprint contenait-il qui n'ont pas été explicitées ?
8. Les dépendances qui nous ont surpris ce sprint auraient-elles pu être identifiées à la planification ?
9. Notre capacité déclarée correspond-elle à notre capacité réelle sur les trois derniers sprints ?
10. Quand avons-nous pour la dernière fois décidé de livrer plus petit pour apprendre plus vite ?

Sur l'implémentation

11. Pouvons-nous tracer chaque story livrée jusqu'à l'exemple ou au critère d'acceptation qui la justifie ?
12. Notre définition de « terminé » est-elle partagée et comprise de la même manière par tous les membres de l'équipe ?
13. La dette technique que nous avons accumulée ce sprint est-elle visible, documentée, et présente dans notre prochaine planification ?
14. Nos revues de code sont-elles des conversations ou des inspections ?
15. Quand avons-nous reporté une livraison parce qu'elle ne protégeait pas encore la valeur clarifiée ? Était-ce la bonne décision ?

Sur l'acceptation holistique

16. Y a-t-il des fonctionnalités livrées dans les derniers mois dont personne ne sait quel signal de valeur elles produisent en production ?
17. Théo (ou l'équivalent PO) est-il impliqué dans la validation avant la livraison — pas seulement après ?
18. Notre DoD a-t-elle évolué au cours des six derniers mois ? Si non, qu'est-ce que cela révèle sur notre apprentissage ?

Sur la connexion avec CARE

19. CARE dispose-t-il de tous les signaux dont il a besoin pour observer ce que nous avons livré ?
20. Y a-t-il des risques résiduels dans notre dernière livraison que CARE ne connaît pas ?
21. Les apprentissages de CARE dans la dernière itération ont-ils enrichi notre planification actuelle ?

Quatrième de couverture

L2 — Le SDLC Réconcilié — Partie B : FORGE

À propos de ce livre

Ce livre est la Partie B du deuxième livre de la collection ToTAL — Réconciliation. Il traite des deux phases centrales du SDLC réconcilié : PLANIFIER et IMPLÉMENTER. Ensemble, elles forment la zone FORGE — la zone où les intentions deviennent des produits construits, intégrés, vérifiés et préparés pour le réel.

FORGE n'est pas une boîte noire de développement. C'est une zone de maîtrise qui chevauche délibérément le portefeuille vers l'amont et l'opérationnel vers l'aval. C'est dans FORGE que la valeur clarifiée par SENSE est transformée en produit éprouvé — avec des preuves progressives, une acceptation holistique, et des signaux préparés pour que CARE puisse observer ce qui compte.

Ce livre s'adresse aux équipes de développement, aux product owners, aux tech leads, aux responsables qualité, et à tous ceux qui veulent réconcilier la rigueur de la construction avec la fluidité de l'apprentissage.

La collection ToTAL — Réconciliation

La collection ToTAL — Réconciliation comprend 21 livres couvrant l'ensemble de la discipline ToTAL, du portefeuille à l'exploitation, de l'élicitation à la confirmation. Chaque livre peut être lu indépendamment ou dans l'ordre de la trajectoire.

ToTAL ne défend ni le modèle en cascade, ni son rejet systématique. Il n'est ni pro-Agile, ni anti-Agile. Il est pro-maîtrise. Sa trajectoire : Clarté → Qualité → Confiance → Excellence durable.

Léo Michel Bélanger

Léo Michel Bélanger est le fondateur d'ORIGINALIUM, basé à Disraeli, QC, Canada. Praticien et penseur en développement logiciel, il a développé la discipline ToTAL en réponse à une conviction simple : les organisations n'ont pas besoin d'une nouvelle méthodologie, elles ont besoin d'une trajectoire de maîtrise qui réconcilie ce qu'elles ont appris avec ce qu'elles doivent encore découvrir.

Auteur de la collection ToTAL — Réconciliation (21 livres), il partage son expérience accumulée auprès d'organisations de toutes tailles confrontées aux défis permanents du développement logiciel : livrer vite sans perdre la qualité, apprendre sans arrêter de construire, et maintenir la valeur visible dans un monde qui change en permanence.

Son travail est ancré dans la conviction que la maîtrise est accessible — non pas comme un état à atteindre, mais comme une trajectoire à habiter avec discipline, lucidité et engagement.

ORIGINALIUM | Disraeli, QC, Canada

Collection ToTAL — Réconciliation · 21 livres · ORIGINALIUM
Images et couverture par Soléane Bélanger
Disraeli, QC, Canada · 2026 et plus